

Support Chat + Notification — Testing Guide (Roman Urdu)

Base URL: `https://<tumhara-host>/api/v1` — har request se pehle apna actual host/port lagana. Har response ek common envelope mein aata hai: `{ success, message, data, errors, statusCode }` — neechе sirf data field dikhaya gaya hai jab tak zaroori na ho.

Contents: 1. Admin/Staff Login — 2. Guest OTP Login — 3. Guest Support Chat — 4. Admin Support Chat — 5. Staff Notifications — 6. Guest Notifications + Device — 7. Generic Device Register — 8. Driver Login — 9. Admin Driver Assign — 10. Driver<->Guest Chat — 11. Broadcast Notification Test

1. Admin/Staff Login

POST /auth/login

Headers:

```
Content-Type: application/json
X-Client-App: portal
```

Request Body:

```
{
  "email": "admin@gms.local",
  "password": "Admin@123!"
}
```

Response (data):

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": {
    "id": "3f2b1a10-...-adminGuid",
    "userName": "admin",
    "firstName": "System",
    "lastName": "Administrator",
    "role": "Administrator",
    "roleCode": "admin",
    "permissions": ["SupportChat.View", "SupportChat.Manage", "Notifications.Send", "..."]
  }
}
```

Yaad rakho: accessToken ab se har admin request ke Authorization: Bearer <accessToken> header mein jayega. user.id is admin ka apna PublicId hai (kam use hoga is guide mein).

2. Guest OTP Login (VIP App)

2.1 — OTP Request

POST /vip-app/auth/request-otp**Request Body:**

```
{ "email": "guest@example.com" }
```

Response (data): true (OTP guest ke email pe chala gaya — dev/test env mein logs ya email inbox check karo).

2.2 — OTP Verify (yahin se guest ka login hota hai)

POST /vip-app/auth/verify-otp**Request Body:**

```
{ "email": "guest@example.com", "code": "123456" }
```

Response (data):

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "expiresAt": "2026-07-31T10:00:00Z",
  "guest": {
    "id": "8b7c...-guestPublicId",
    "firstName": "Ali",
    "lastName": "Khan",
    "email": "guest@example.com",
    "organization": "ACME",
    "tier": "VIP"
  }
}
```

Yaad rakho: accessToken ab se guest ki har request mein Authorization: Bearer <guestAccessToken> jayega. guest.id (Guest ka apna PublicId) admin side ke "start-or-reply by guest" endpoint mein use hoga (Step 4.3).

3. Guest Support Chat (guest apna token use karega)

3.1 — Guest apna message bheje (support ko)

POST /support-chat/my/messages (Authorization: Bearer <guestAccessToken>)

Request Body:

```
{
  "body": "Mujhe apni flight ke baare mein poochna hai",
  "attachmentUrl": null,
  "attachmentType": null
}
```

Response (data):

```
{
  "id": "c1d2...-messagePublicId",
  "conversationId": "e5f6...-conversationPublicId",
  "body": "Mujhe apni flight ke baare mein poochna hai",
  "fromGuest": true,
  "sentAt": "2026-07-30T18:00:00Z",
  "isRead": false,
  "senderName": null
}
```

Yaad rakho: conversationId yahan se mil gaya — agar admin side se seedha conversationId chahiye to yehi use hoga (Step 4.2). Agar conversation pehli baar bana hai to yeh naya id hai, warna purana wala hi wapis aayega (ek guest ka sirf ek AdminSupport thread hota hai).

3.2 — Guest apni conversations dekhe

GET /support-chat/my/conversations

Response (data — list, usually 1 item):

```
[{
  "id": "e5f6...-conversationPublicId",
  "guestId": "8b7c...-guestPublicId",
  "guestName": "Ali Khan",
  "status": "Open",
  "lastMessagePreview": "Mujhe apni flight ke baare mein poochna hai",
  "unreadCount": 0
}]
```

3.3 — Guest apne messages dekhe (pagination ke saath)

GET /support-chat/my/messages?pageNumber=1&pageSize=50

Response (data):

```
{
  "items": [ { "id": "...", "body": "...", "fromGuest": true, "sentAt": "..." } ],
  "totalCount": 3,
  "pageNumber": 1,
  "pageSize": 50
}
```

3.4 — Guest apne unread messages "read" mark kare

POST /support-chat/my/messages/read

Request Body: khaali (koi body nahi chahiye)

Response (data): true

4. Admin Support Chat (admin token, permission: SupportChat.View / SupportChat.Manage)

4.1 — Admin: sab conversations ki list (inbox)

GET /support-chat/conversations?pageNumber=1&pageSize=20&onlyUnread=false (Authorization: Bearer <adminAccessToken>)

Response (data):

```
{
  "items": [{
    "id": "e5f6...-conversationPublicId",
    "guestId": "8b7c...-guestPublicId",
    "guestName": "Ali Khan",
    "guestEmail": "guest@example.com",
    "status": "Open",
    "lastMessagePreview": "Mujhe apni flight ke baare mein poochna hai",
    "unreadCount": 1
  }],
  "totalCount": 1, "pageNumber": 1, "pageSize": 20
}
```

Yaad rakho: items[0].id (conversation PublicId) — Step 4.2, 4.4, 4.5, 4.6 mein use hoga. items[0].guestId (Guest PublicId) — Step 4.3 mein use hoga.

4.2 — Admin: ek conversation ke messages dekhe

GET /support-chat/conversations/{conversationId}/messages?pageNumber=1&pageSize=50

{conversationId} = Step 4.1 ka items[0].id (ya Step 3.1 ka conversationId)

Response (data): same shape jaisa Step 3.3

4.3 — Admin: guest ko reply kare (conversation na bhi bana ho to bhi chalega)

POST /support-chat/conversations/by-guest/{guestId}/messages

{guestId} = Step 2.2 ka guest.id (Guest PublicId)

Request Body:

```
{
  "body": "Aap ki flight details bhej di gayi hain, check karein.",
  "attachmentUrl": null,
  "attachmentType": null
}
```

Response (data): naya message object (jaisa 3.1) — fromGuest: false, senderName: "System Administrator"

Yeh endpoint tab kaam aata hai jab admin ke paas already conversationId na ho — sirf guestId se seedha reply/start ho jata hai.

4.4 — Admin: ek existing conversation pe reply (conversationId maloom ho)

POST /support-chat/conversations/{conversationId}/messages

Request Body: same shape jaisa 4.3

4.5 — Admin: conversation ke unread messages "read" mark kare

POST /support-chat/conversations/{conversationId}/read

Response (data): true

4.6 — Admin: conversation close / reopen kare

POST /support-chat/conversations/{conversationId}/close

POST /support-chat/conversations/{conversationId}/reopen

Response (data): true dono mein

5. Notifications — Staff/Admin side

5.1 — Apni notifications ki list

GET /notifications?pageNumber=1&pageSize=20&isRead=false

Response (data):

```
{
  "items": [{
    "id": "f1a2...-notificationPublicId",
    "userId": "3f2b...-adminGuid",
    "title": "New support message",
    "message": "Mujhe apni flight ke baare mein poochna hai",
    "type": "support_message",
    "read": false,
    "createdAt": "2026-07-30T18:00:05Z",
    "redirectUrl": "/admin/support-chat/e5f6...-conversationPublicId"
  }],
  "totalCount": 1
}
```

Yaad rakho: items[0].id — Step 5.3/5.4 mein use hoga.

5.2 — Unread count

GET /notifications/count

Response (data): 1 (int)

5.3 — Single notification read/unread mark kare

PUT /notifications/{id}/mark-read

PUT /notifications/{id}/mark-unread

{id} = Step 5.1 ka items[0].id

5.4 — Sab mark-read / delete

PUT /notifications/mark-all-read

DELETE /notifications/{id}

5.5 — Admin kisi ko/kisi role ko notification bheje

POST /notifications/send (permission: Notifications.Send)

Request Body — sirf EK target field bharo (chahe koi bhi):

```
// (A) specific users ko
{ "title": "Test", "message": "Hello", "userIds": ["<user PublicId>"] }

// (B) role ko (sab active users jin ka role match ho)
{ "title": "Test", "message": "Hello", "roleCode": "guest-relations-manager" }

// (C) permission wale sab ko
{ "title": "Test", "message": "Hello", "permissionCode": "SupportChat.Manage" }

// (D) sab active staff ko
{ "title": "Test", "message": "Hello", "broadcast": true }

// (E) specific guests ko
{ "title": "Test", "message": "Hello", "guestIds": ["<guest PublicId>"] }

// (F) sab guests ko (jinka NotificationsEnabled=true)
{ "title": "Test", "message": "Hello", "broadcastGuests": true }
```

Response (data):

```
{ "recipientCount": 1, "targetType": "Users" }
```

6. Notifications — Guest side (VIP app token)

6.1 — Guest apni notifications dekhe

GET /notifications/guest?pageNumber=1&pageSize=20

Response (data): list, shape jaisa 5.1 (bina pagination wrapper ke — seedha array)

6.2 — Guest unread count

GET /notifications/guest/count

6.3 — Guest single/all read mark kare

PUT /notifications/guest/{id}/mark-read

PUT /notifications/guest/mark-all-read

6.4 — Guest apna push-device register kare (purana route, VIP app compatibility ke liye)

POST /notifications/guest/devices**Request Body:**

```
{ "token": "fcm-device-token-abc123", "platform": "android" }
```

Response (data): true

7. Device register/deregister — generic (koi bhi token: staff/driver/guest)

POST /notifications/devices**Request Body:**

```
{ "token": "fcm-device-token-abc123", "platform": "web" }
```

Response (data): true**DELETE** /notifications/devices?token=fcm-device-token-abc123**Response (data):** true

Naya generic endpoint hai — staff, driver, guest — koi bhi apna token bhej ke apna device register kar sakta hai. Firebase configured ho to yahan se real push jayegi (dekhein appsettings.json ka Firebase section).

8. Driver Login

POST /auth/login**Headers:**

```
Content-Type: application/json
X-Client-App: driver-app
```

Request Body:

```
{ "email": "driver1@gms.local", "password": "Driver@123!" }
```

Response (data):

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": {
    "id": "9a8b...-driverUserPublicId",
    "role": "Driver",
    "roleCode": "driver",
    "driverDetails": { "id": "...", "driverType": "Fixed", "licenseNumber": "..." }
  }
}
```

Yaad rakho: user.id (driver ka apna User PublicId) — yeh **guest** ke liye recipientUserId banega Step 10.2 mein.

9. Admin: Guest ko Driver assign kare (Driver<->Guest chat se pehle zaroori)

POST /transportation/guest/{guestId}/drivers (admin token, permission: Transportation.Assign)

{guestId} = Step 2.2 ka guest.id

Request Body:

```
{ "driverIds": ["<DriverProfile PublicId>"] }
```

Dhyan se: driverIds mein **DriverProfile ka PublicId** jata hai, driver ke User PublicId se alag hai. Confirm karne ke liye:

GET /transportation/guest/{guestId}/drivers se assigned drivers ki list mil jayegi.

10. Driver <-> Guest Chat (naya endpoint)

Ek hi endpoint dono directions handle karta hai: **POST** /support-chat/driver-guest/messages

10.1 — Guest driver ko message bheje (role se target — assignment pool se auto-resolve)

POST /support-chat/driver-guest/messages (Authorization: Bearer <guestAccessToken>)

Request Body:

```
{
  "recipientRole": "driver",
  "body": "Aap kahan tak pohonche?",
  "attachmentUrl": null,
  "attachmentType": null
}
```

Response (data):

```
{
  "id": "...-messagePublicId",
  "conversationId": "...-driverGuestConversationPublicId",
  "body": "Aap kahan tak pohonche?",
  "fromGuest": true,
  "sentAt": "2026-07-30T19:00:00Z"
}
```

Zaroori: guest ke paas **exactly ek** assigned driver hona chahiye (Step 9), warna error aayega ("no driver assigned" ya "more than one driver assigned — specify recipientUserId").

10.2 — Driver guest ko reply kare (specific guest, recipientUserId se)

POST /support-chat/driver-guest/messages (Authorization: Bearer <driverAccessToken>)

Request Body:

```
{
  "recipientUserId": "<guest ka linked User PublicId>",
  "body": "5 minute mein pohonch jaunga",
  "attachmentUrl": null,
  "attachmentType": null
}
```

Gap (abhi koi endpoint yeh directly nahi deta): guest ka apna User.PublicId (Guest.PublicId se alag) kahin bhi response mein expose nahi hota. Testing ke liye database se nikal lo:

```
SELECT u.PublicId
FROM Users u
JOIN Guests g ON g.UserId = u.Id
WHERE g.PublicId = '<guest.id ya Step 2.2 wala guestId>';
```

Yehi PublicId upar recipientUserId mein daalo. (Follow-up: agar yeh feature production mein driver-initiated chat frequently use hogi, to koi admin/driver endpoint isay expose karna chahiye.)

Response (data): same shape jaisa 10.1, fromGuest: false

Yaad rakho: dobara same recipientUserId/recipientRole se bhejo to same conversation reuse hogi, nayi nahi banegi.

10.3 — Validation cases jo test karni hain

Scenario	Expected
Staff (portal) token se yeh endpoint call karo	403 Forbidden — "Only drivers and guests can use this endpoint"
recipientUserId galat/na-mojood Guid	404 — "Recipient not found"
Recipient IsActive=false	400 — "Recipient is inactive"
Guest recipientUserId ek dusre guest ko de (driver ke bajaye)	400 — "Invalid recipient — expected a driver"
Driver bina recipientUserId ke, sirf recipientRole="guest" bheje	400 — driver ko hamesha recipientUserId dena zaroori hai
Body aur attachment dono khaali	400 — "Message body or an attachment is required"

11. End-to-end quick sanity flow (sab ek saath)

#	Action	Endpoint	ID capture
1	Admin login	POST /auth/login	adminAccessToken
2	Guest OTP verify	POST /vip-app/auth/verify-otp	guestAccessToken, guest.id
3	Guest support message bheje	POST /support-chat/my/messages	conversationId
4	Admin inbox dekhe	GET /support-chat/conversations	confirm conversationId, guestId
5	Admin reply kare	POST /support-chat/conversations/by-guest/{guestId}/messages	-
6	Guest apna unread notification dekhe	GET /notifications/guest	notification id
7	Driver login	POST /auth/login (X-Client-App: driver-app)	driverAccessToken, driver user.id
8	Admin driver ko guest assign kare	POST /transportation/guest/{guestId}/drivers	-
9	Guest driver ko message bheje	POST /support-chat/driver-guest/messages (recipientRole=driver)	driverGuest conversationId
10	Driver reply kare	POST /support-chat/driver-guest/messages (recipientUserId=guest's linked User PublicId)	-

Reference: docs/guest-user-merge-and-chat-refactor.md — full architecture aur DB changes ke liye.